

Disciplina: Aprendizado Profundo

Aula 6: Laboratório 2 - Otimizadores na Prática

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação, Universidade Federal do
Ceará (MDCC / UFC)

1 de Outubro de 2025

Roteiro da Aula de Hoje

1 Revisão dos Conceitos de Otimização

2 Exercícios Práticos

3 Próximos Passos

Recapitulando a Aula 5 I

O Desafio da Otimização

- A paisagem de custo de uma rede neural é **não-convexa**, cheia de mínimos locais, pontos de sela e platôs.
- O SGD padrão, embora simples, pode ser lento e oscilar em vales estreitos.

Soluções que Vimos

- **Momento:** Adiciona "inércia" para acelerar em direções consistentes e amortecer oscilações.
- **Taxas de Aprendizado Adaptativas (RMSProp/Adam):** Ajustam o tamanho do passo para cada parâmetro individualmente, baseando-se na história de seus gradientes.

Objetivo do Laboratório de Hoje I

Da Teoria à Prática

Hoje, vamos implementar e comparar visualmente o desempenho de diferentes otimizadores. O objetivo é construir a intuição sobre como cada um deles navega a paisagem de custo.

Objetivo do Laboratório de Hoje II

As Ferramentas

- 1 **NumPy:** Vamos estender nosso framework "do zero" para incluir os otimizadores Momento e Adam.
- 2 **PyTorch:** Vamos ver como usar os otimizadores já implementados e otimizados do PyTorch.
- 3 **Visualização:** Usaremos um dataset 2D para plotar as fronteiras de decisão e as curvas de custo, comparando a velocidade e a qualidade da convergência.

Exercício 1: Implementando Otimizadores em NumPy I

A Tarefa

No notebook, você encontrará o esqueleto das classes 'MomentumSGD' e 'Adam'. Sua tarefa é preencher os métodos 'step' para cada um, seguindo as regras de atualização que vimos na aula.

Regra de Atualização: SGD com Momento

$$\mathbf{v}_{t+1} \leftarrow \beta \mathbf{v}_t + (1 - \beta) \nabla J(\theta_t)$$

$$\theta_{t+1} \leftarrow \theta_t - \eta \mathbf{v}_{t+1}$$

Exercício 1: Implementando Otimizadores em NumPy II

Regra de Atualização: Adam

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla J(\theta_t)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) [\nabla J(\theta_t)]^{\odot 2}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

Exercício 2: Comparando a Convergência I

O Experimento

Treine a mesma arquitetura de MLP no dataset "Duas Luas" usando os três otimizadores que você implementou (SGD, Momento, Adam).

Análise Visual

Plote as curvas de custo (loss vs. épocas) para os três otimizadores em um único gráfico.

- O que você observa sobre a velocidade de convergência de cada um?
- O Adam realmente converge mais rápido?
- A curva do SGD é mais "ruidosa"?

Exercício 3: Otimizadores em PyTorch I

A Simplicidade de um Framework

Agora, vamos replicar o mesmo experimento usando PyTorch. A tarefa é construir o mesmo MLP e treiná-lo com os otimizadores 'torch.optim.SGD', 'torch.optim.SGD' (com momentum) e 'torch.optim.Adam'.

O Ponto-Chave

Observe como a estrutura do loop de treinamento permanece a mesma. A única coisa que muda é a linha onde você instancia o otimizador:

Isso demonstra o poder de uma boa abstração de software, que é o que nosso mini-framework tentou emular.

Encerramento e Próximos Passos I

Resumo do Laboratório

- Implementamos os otimizadores Momento e Adam do zero, entendendo sua mecânica interna.
- Comparamos visualmente a dinâmica de convergência do SGD, Momento e Adam.
- Vimos como utilizar a API de otimizadores do PyTorch para realizar os mesmos experimentos de forma muito mais simples.

Encerramento e Próximos Passos II

Para a Próxima Aula

- **Tópico:** Redes Neurais Convolucionais (CNNs) e Invariâncias.
- **Objetivo:** Sair do mundo dos dados tabulares e MLPs para o domínio de imagens, introduzindo as operações de convolução e pooling. Ideia de implementar invariâncias na rede.

Referências I

-  Bach, Francis (2025). *Learning Theory from First Principles*.
URL: https://www.di.ens.fr/~fbach/ltpf_book.pdf.
-  Belkin, Mikhail et al. (2019). “Reconciling modern machine-learning practice and the classical bias–variance trade-off”. Em: *Proceedings of the National Academy of Sciences* 116.32, pp. 15849–15854.
-  Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.
-  Bishop, Christopher M. e Hugh Bishop (2023). *Deep Learning: Foundations and Concepts*. Springer.
-  Buchanan, Sam et al. (ago. de 2025). *Learning Deep Representations of Data Distributions*.
<https://ma-lab-berkeley.github.io/deep-representation-learning-book/>. Online.

Referências II

-  Cybenko, George (1989). “Approximation by superpositions of a sigmoidal function”. Em: *Mathematics of control, signals and systems* 2.4, pp. 303–314.
-  Domke, Justin (2011). *Automatic Differentiation and Neural Networks*. Lecture Notes, Statistical Machine Learning. URL: https://people.cs.umass.edu/~domke/courses/sml2011/08autodiff_nnets.pdf.
-  Gefter, Amanda (fev. de 2015). “The Man Who Tried to Redeem the World with Logic”. Em: *Nautilus*. URL: <https://nautil.us/the-man-who-tried-to-redeem-the-world-with-logic-235253/>.
-  Grünwald, Peter D. (2007). *The Minimum Description Length Principle*. The MIT Press.

Referências III

-  Hastie, Trevor et al. (2022). “Surprises in high-dimensional ridgeless least squares interpolation”. Em: *The Annals of Statistics* 50.2, pp. 949–986.
-  Hochreiter, Sepp e Jürgen Schmidhuber (1997). “Flat minima”. Em: *Neural computation* 9.1, pp. 1–42.
-  Jordan, Michael I (2019). “Artificial intelligence—the revolution hasn’t happened yet”. Em: *Harvard Data Science Review* 1.1.
-  Keskar, Nitish Shirish et al. (2017). “On large-batch training for deep learning: Generalization gap and sharp minima”. Em: *International Conference on Learning Representations (ICLR)*.
-  Kingma, Diederik P. e Jimmy Ba (2015). “Adam: A Method for Stochastic Optimization”. Em: *ICLR 2015*.
-  Krause, Andreas e Jonas Hübner (2025). *Probabilistic Artificial Intelligence*. arXiv: 2502.05244 [cs.AI].

Referências IV

-  Liu, Yuxi (jan. de 2024). *The Perceptron Controversy*. Website. URL: <https://yuxi.ml/essays/posts/perceptron-controversy/> (acesso em 10/09/2025).
-  Martins, André FT e Ramón F Astudillo (2016). “From softmax to sparsemax: A sparse model of attention and multi-label classification”. Em: *International conference on machine learning*. PMLR, pp. 1614–1623.
-  McCulloch, Warren S. e Walter Pitts (1943). “A Logical Calculus of the Ideas Immanent in Nervous Activity”. Em: *The Bulletin of Mathematical Biophysics* 5.4, pp. 115–133. DOI: [10.1007/bf02478259](https://doi.org/10.1007/bf02478259).

Referências V

-  Mercer, James (1909). “Functions of positive and negative type, and their connection with the theory of integral equations”. *Em: Philosophical transactions of the Royal Society of London. Series A* 209.441-458, pp. 415–446.
-  Minsky, Marvin e Seymour A Papert (1969). *Perceptrons: An introduction to computational geometry*. MIT press.
-  Montufar, Guido F et al. (2014). “On the number of linear regions of deep neural networks”. *Em: Advances in neural information processing systems* 27.
-  Murphy, Kevin Patrick (2023). *Probabilistic Machine Learning: Advanced Topics*. The MIT Press.
-  Nesterov, Yurii (2004). *Introductory lectures on convex optimization: A basic course*. Vol. 87. Springer Science & Business Media.

Referências VI

-  Oja, Erkki (1982). “A simplified neuron model as a principal component analyzer”. Em: *Journal of mathematical biology* 15.3, pp. 267–273.
-  Paes, Lucas Monteiro et al. (2023). “On the Inevitability of the Rashomon Effect”. Em: *IEEE Journal on Selected Areas in Information Theory* 4.2, pp. 416–427. DOI: 10.1109/JSAIT.2023.3287661.
-  Prince, Simon J.D. (2023). *Understanding Deep Learning*. The MIT Press.
-  Rasmussen, Carl Edward e Christopher K. I. Williams (2006). *Gaussian processes for machine learning*. The MIT Press.
-  Rosenblatt, Frank (1958). “The perceptron: a probabilistic model for information storage and organization in the brain.”. Em: *Psychological review* 65.6, p. 386.

Referências VII

-  Rudin, Cynthia et al. (2024). “Position: amazing things come from having many good models”. Em: *ICML*.
-  Schölkopf, Bernhard e Alexander J Smola (2002). *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
-  Shalev-Shwartz, Shai e Shai Ben-David (2014). *Understanding machine learning: From theory to algorithms*. Cambridge university press.
-  Shannon, C. E. (1988). “Programming a computer for playing chess”. Em: *Computer Chess Compendium*. Berlin, Heidelberg: Springer-Verlag, pp. 2–13. ISBN: 0387913319.
-  Shannon, Claude E. (1948). *A Mathematical Theory of Communication*. Vol. 27, pp. 379–423, 623–656.

Referências VIII

-  Turing, Alan M. (1969). “Intelligent Machinery”. Em: *Machine Intelligence 5*. Ed. por Bernard Meltzer e Donald Michie. Edinburgh University Press, pp. 3–23.
-  Vapnik, Vladimir (1998). *Statistical learning theory*. Wiley.
-  Vieira, Tim (ago. de 2017). *Backprop is not just the chain rule*. Blog Post. URL: <https://timvieira.github.io/blog/post/2017/08/18/backprop-is-not-just-the-chain-rule/> (acesso em 15/09/2025).